indiXpert

State Management using Redux

React JS

What is Redux & related terms?

Introduction

Understanding **Redux**, **React-Redux**, and **Redux Toolkit (RTK)**

Redux : Redux is the core, standalone JavaScript library for handling predictable state management in any JavaScript app, like **Angular, Vue, React and more...**

Features:

- Implements the Flux pattern (with actions, reducers, and a single global store).
- Uses plain JavaScript (no React-specific features).
- Powerful and highly customizable.

Redux Toolkit (RTK) : Redux Toolkit is the officially recommended modern approach to writing Redux logic.Redux Toolkit, while commonly associated with React due to its popularity in the React ecosystem, is a standalone library that can be used with any UI layer or framework. This means it can be implemented in applications built with **Vue, Angular etc...**

Features:

- Wraps around the Redux core library to simplify standard Redux tasks.
- Provides helper APIs (like configureStore, createSlice, createAsyncThunk) that reduce boilerplate.
- Built-in support for common middleware (like redux-thunk), DevTools, and state immutability via Immer.
- Encourages best practices with built-in patterns for typical use cases.
- Reduces the need for multiple configuration files and code repetition.

React-Redux : React-Redux is the official binding library that connects Redux with **React applications**.

Features:

- Provides React components (<Provider>) and hooks (useSelector, useDispatch).
- Allows React components to access Redux state and dispatch actions easily.
- Handles the store updates efficiently for React.

How Redux States are working?

Installation

```
npm install react-redux @reduxjs/toolkit
```

Steps to create and use redux context.

- **Create or Configure Store:**
  Use configureStore() from reduxjs toolkit to create a state management store object for sharing data globally.
- **Set Store Provider:**
  Wrap components needing access with a <Provider>, passing the data via the store prop.
- **Create Slices :** create required slices using createSlice(), it's nothing but works like **useReducer** hook.
- **Use or Manage Store Data:**
  Use the useDispatch() and useSelector() hooks in functional components to execute any **action** and **fetch current data** from store respectivly.

Create Store

Use **configureStore**() to create and configure a store for redux to manage various stats globally.

/src/store/Store.jsx. (Method 1)

```
import { configureStore } from '@reduxjs/toolkit';
export const store = configureStore({
  reducer : { .....list of all reducers..... },
});
```

/src/store/Store.jsx (Method 2, it's deprecated)

```
import { createStore,combineReducers } from '@reduxjs/toolkit';
export const store = createStore(
    combineReducers({
            .....list of all reducers.....
    })
);
```

How Redux States are working? (steps)

Set Store Provider

Wrap components needing access with a **<Provider>**, importing from react-redux and passing the complete store object with **store** parameter.

/src/App.jsx

```
import { RouterProvider } from 'react-router-dom';
import { router } from "./router/DataModeRouter";
import { Provider } from "react-redux";
import { store } from "./store/store";

const App = () => {
    return (
        <Provider store={store}>
            <RouterProvider router={router} />
        </Provider>
    );
};
export default App;
```

How Redux States are working? (steps)

createSlice

Slices in Redux are nothing but similar to our useReducer (Context API) with Redux syntax. Creating a slice requires a string **name** to identify the slice, an **initial state value**, and one or more **reducer functions** to define how the state can be updated. Once a slice is created, we can **export the generated Redux action** creators and the reducer function for the whole slice. We can call slices as features, reducers or actions.

/src/store/actions/wishlistSlice.jsx

```
import { createSlice } from "react-router-dom";

export const wishlistSlice = createSlice({
    name: "wishlist",
    initialState : { userid:1, wishlistItems: [ { id: uuid(), productID: 1 }, { id: uuid(), productID: 2 },
    reducers: {
        addToWishlist: (state,action) => {
            let newItem = {
                id: uuid(),
                productID: action.payload.id,
                title: action.payload.title,
                price: action.payload.price,
                thumbnail: action.payload.thumbnail,
            };
            state.wishlistItems = [...state.wishlistItems, newItem];
        },
        removeFromWishlist: (state,action) => {
            state.wishlistItems = state.wishlistItems.filter( (product) => product.id !== action.payload  );
        },
    },
});
export const { addToWishlist, removeFromWishlist } = wishlistSlice.actions;
export default wishlistSlice.reducer;
```

How Redux States are working? (steps)

Import and include created slice in store

Once slice has been created you need to import and add in the list of reducers of the store which is created at **/src/store/Store.jsx**

/src/store/Store.jsx

```
import { configureStore } from "@reduxjs/toolkit";
import { wishlistSlice } from "./actions/WishlistSlice";

export const store = configureStore({
    reducers: {
        wishlist: wishlistSlice.reducer,
    },
});
```

How Redux States are working? (steps)

Update slice data using actions

The code shown here is for the basic structure understanding that shows how to use actions created in redux slice.

Considering our example we want to use the created slice's action for "Add To Wishlist" button which is located in Products.jsx file, so you can use addToWishlist() action as shown below.

/src/pages/Products.jsx

```
import { addToWishlist } from "../store/actions/WishlistSlice";
import { useDispatch } from "react-redux";

const Products = () => {
        const dispatch = useDispatch();
        const handleButtonClick = (product) => {
            dispatch(addToWishlist(product));
        };
        return (
                ....
                <Button size="sm" onClick={() => handleButtonClick(product)} variant="outline-success" >
                        Add to Wishlist
                </Button>
                ....
        );
};
export default Products;
```

How Redux States are working? (steps)

Show slice data using useSelector Hook

Considering our example we want to use the created slice's action for "Add To Wishlist" button which is located in Products.jsx file, so you can use addToWishlist() action as shown below.

/src/pages/Wishlist.jsx

```
import { removeFromWishlist } from "../store/actions/WishlistSlice";
import { useDispatch, useSelector } from "react-redux";

const Wishlist = () => {
        const dispatch = useDispatch();
        const { wishlistItems } = useSelector((state) => state.wishlist);
        const handleRemoveItem = (itemID) => {  dispatch(removeFromWishlist(itemID));  };
        return (
                .... {wishlistItems?.map((item) => {
                ....
                <Button size="sm" onClick={() => handleRemoveItem(item?.id)} variant="outline-danger" >Remove
                </Button>
                ....
        );
};
export default Wishlist;
```

The code shown here is for the basic structure understanding that shows how remove button will call remove method defined in redux store.

Below code will be helpful to get all store data.
console.log("Redux store:", store.getState()); // to get current state data

Exercise

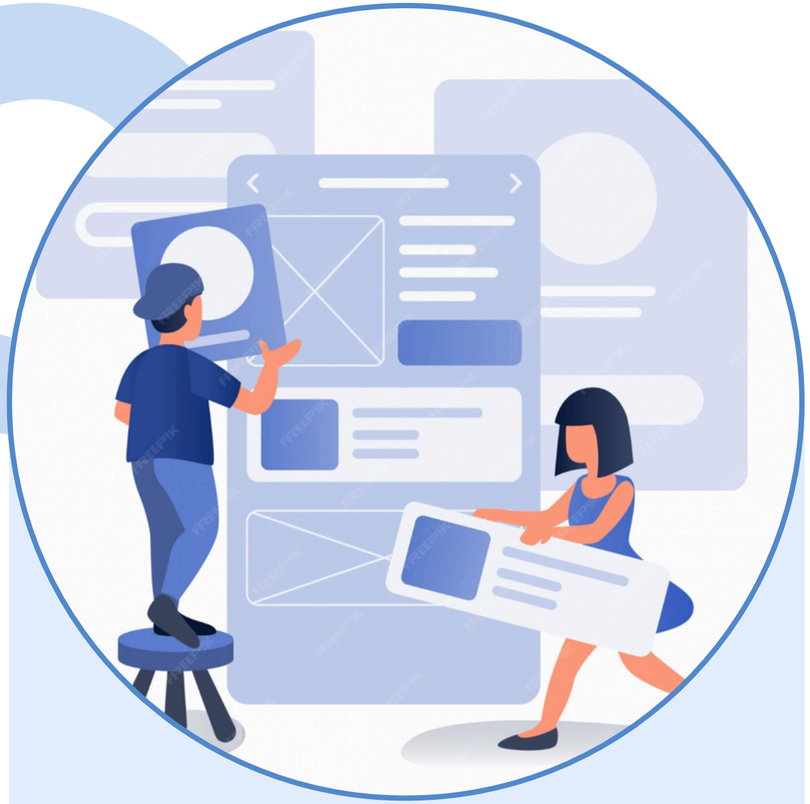
Create new project setup named **react-ecommerce** and implement the eCommerce functionalities as listed below, using **react-redux** and **@reduxjs/toolkit** (**RTK**)

- Categories Listing
- Clicking on Category open products list of the same or selected category, in which user can Add To Wishlist and Add to Cart any product, which contains details like Product image, name, add to wishlist button, quantity box and add to cart button..
- Product Detail page, same facility like product listing, additionally show products slider and description.
- For Products list and product pages...
  - On clicking Add to wishlist, item should be added to wishlist, if item is already added, show notification it's already added else show notification "Item successfully added in wishlist", also add facility to remove item from wishlist and move it to cart using Add to Cart button.
  - On clicking Add to cart, item should be added to Cart, if item is already added, update specified quantities.
- Create a Complete shopping cart functionalities as per below.
  - Add to cart with quantities.
  - Update cart quantities.
  - Remove product from the cart.
  - Total cart amount with the calculation of subtotal, Fixed amount of shipping cost and and fixed percentage tax.
  - Checkout page that shows order summary and button to place order.

For Product and categories data you can use below JSON data https://dummyjson.com/products

Conclusion

We have learn how to manage global state using Redux and based on that we are now able to develop application like eCommerce, having a functionalities like add to cart, add to wishlist, compare products etc...



THANK YOU!

indiXpert